DEIMOS ADCS Quaternion Kinematics

Vivek Swamy
IIT Indore

20 June 2026

1 Introduction

The attitude of a spacecraft describes its orientation relative to an inertial reference frame. In order to simulate spacecraft motion, a mathematical representation of orientation and a corresponding propagation law are required.

Although Euler angles provide an intuitive description of orientation, they suffer from singularities known as gimbal lock. Furthermore, their differential equations become increasingly complex for large rotations.

Quaternions provide a compact, singularity-free representation of orientation and are therefore widely used in modern Attitude Determination and Control Systems (ADCS). The objective of this document is to derive the quaternion kinematic equation used in the simulator.

2 Quaternion Representation

2.1 Euler's Rotation Theorem

Any orientation of a rigid body can be reached by one rotation, by some angle ϕ, about one fixed axis $\hat{\boldsymbol{e}}$. This single fact is what makes the axis-angle and quaternion representations possible.

2.2 Axis-Angle Representation

The pair $(\hat{\boldsymbol{e}}, \phi)$ describes orientation directly, but it cannot be updated using a simple linear equation involving angular velocity. The quaternion below fixes this.

2.3 Quaternion Definition

$$\boldsymbol{q} = \begin{bmatrix} q_0 \\ q_1 \\ q_2 \\ q_3 \end{bmatrix} = \begin{bmatrix} \cos(\phi/2) \\ e_1 \sin(\phi/2) \\ e_2 \sin(\phi/2) \\ e_3 \sin(\phi/2) \end{bmatrix}$$

2.4 Scalar and Vector Components

$$\boldsymbol{q} = \begin{bmatrix} q_0 \\ \boldsymbol{q}_v \end{bmatrix}, \qquad q_0 = \cos(\phi/2), \qquad \boldsymbol{q}_v = \hat{\boldsymbol{e}} \sin(\phi/2)$$

q_0 is called the scalar part, and $\boldsymbol{q}_v$ is the vector part. This split is used in Section 4 to derive how the quaternion changes over time.

2.5 Unit Quaternion Constraint

$$q_0^2 + q_1^2 + q_2^2 + q_3^2 = 1$$

This means $\boldsymbol{q}$ always has unit length. Numerical integration can slowly drift away from this, so python renormalizes the quaternion at every step.

3 Angular Velocity Representation

The rotational state of a spacecraft is characterized by its angular velocity vector

$$\boldsymbol{\omega} = \begin{bmatrix} \omega_x \\ \omega_y \\ \omega_z \end{bmatrix}.$$

The angular velocity vector specifies the instantaneous rate of rotation of the spacecraft body frame with respect to an inertial reference frame.

In the simulator, the rotational dynamics module computes the evolution of $\boldsymbol{\omega}$ from Euler's rigid body equations. The role of quaternion kinematics is then to determine how the spacecraft attitude changes in response to this angular velocity.

4 Derivation of Quaternion Kinematics

Over a very small time step dt, the body rotates by a small angle $d\phi = \|\boldsymbol{\omega}\|\, dt$ about the axis $\hat{\boldsymbol{e}} = \boldsymbol{\omega}/\|\boldsymbol{\omega}\|$. This small rotation corresponds to a small quaternion change:

$$\delta\boldsymbol{q} = \begin{bmatrix} \cos(d\phi/2) \\ \hat{\boldsymbol{e}} \sin(d\phi/2) \end{bmatrix} \approx \begin{bmatrix} 1 \\ \frac{1}{2}\boldsymbol{\omega}\, dt \end{bmatrix}$$

using the standard small-angle approximations $\cos(d\phi/2) \approx 1$ and $\sin(d\phi/2) \approx d\phi/2$.

The new quaternion is found by multiplying this small change with the current one:

$$\boldsymbol{q}(t + dt) = \delta\boldsymbol{q} \otimes \boldsymbol{q}(t)$$

For two quaternions $\boldsymbol{p} = [p_0,\, \boldsymbol{p}_v]$ and $\boldsymbol{q} = [q_0,\, \boldsymbol{q}_v]$, multiplication is defined as

$$\boldsymbol{p} \otimes \boldsymbol{q} = \begin{bmatrix} p_0 q_0 - \boldsymbol{p}_v \cdot \boldsymbol{q}_v \\ p_0 \boldsymbol{q}_v + q_0 \boldsymbol{p}_v + \boldsymbol{p}_v \times \boldsymbol{q}_v \end{bmatrix}$$

Substituting $\delta\boldsymbol{q}$ and keeping only first-order terms in dt gives

$$\boldsymbol{q}(t + dt) = \boldsymbol{q}(t) + \frac{1}{2} \begin{bmatrix} -\boldsymbol{\omega} \cdot \boldsymbol{q}_v \\ q_0 \boldsymbol{\omega} + \boldsymbol{\omega} \times \boldsymbol{q}_v \end{bmatrix} dt$$

Letting $dt \to 0$ gives the rate of change of each part of the quaternion:

$$\dot{q}_0 = -\frac{1}{2}\boldsymbol{\omega} \cdot \boldsymbol{q}_v, \qquad \dot{\boldsymbol{q}}_v = \frac{1}{2}\left(q_0 \boldsymbol{\omega} + \boldsymbol{\omega} \times \boldsymbol{q}_v\right)$$

Together, these give the main result of this note:

$$\boxed{\dot{\boldsymbol{q}} = \frac{1}{2}\,\Omega(\boldsymbol{\omega})\,\boldsymbol{q}}$$

This equation is simple to integrate: it is linear in $\boldsymbol{q}$, has no trigonometric terms, and works for any orientation.

5 Matrix Form

Writing $\boldsymbol{\omega} = [\omega_x,\ \omega_y,\ \omega_z]^T$, the matrix $\Omega(\boldsymbol{\omega})$ is

$$\Omega(\boldsymbol{\omega}) = \begin{bmatrix} 0 & -\omega_x & -\omega_y & -\omega_z \\ \omega_x & 0 & \omega_z & -\omega_y \\ \omega_y & -\omega_z & 0 & \omega_x \\ \omega_z & \omega_y & -\omega_x & 0 \end{bmatrix}$$

so the full equation looks like this:

$$\begin{bmatrix} \dot{q}_0 \\ \dot{q}_1 \\ \dot{q}_2 \\ \dot{q}_3 \end{bmatrix} = \frac{1}{2} \begin{bmatrix} 0 & -\omega_x & -\omega_y & -\omega_z \\ \omega_x & 0 & \omega_z & -\omega_y \\ \omega_y & -\omega_z & 0 & \omega_x \\ \omega_z & \omega_y & -\omega_x & 0 \end{bmatrix} \begin{bmatrix} q_0 \\ q_1 \\ q_2 \\ q_3 \end{bmatrix}$$

The first row gives $\dot{q}_0$, and the last three rows give $\dot{\boldsymbol{q}}_v$, matching what we derived in Section 4.

6 Numerical Implementation

At each simulation step, python builds $\Omega(\boldsymbol{\omega})$ from the current angular velocity and multiplies it with the current quaternion:

```
import numpy as np


class QuaternionKinematics:

    def quaternion_dot(self, q, omega):

        """
        Quaternion Propogation Equation:

        q_dot = 0.5 * Omega(omega) * q
        where Omega(omega) is the skew-symmetric matrix of angular velocity omega:

        Omega(omega) =

                        [ 0 -wx -wy -wz ]
                        [ wx 0 wz -wy ]
                        [ wy -wz 0 wx ]
                        [ wz wy -wx 0 ]


        """
        wx = omega[0]
        wy = omega[1]
        wz = omega[2]

        Omega = np.array([
            [0, -wx, -wy, -wz],
            [wx, 0, wz, -wy],
            [wy, -wz, 0, wx],
            [wz, wy, -wx, 0 ]
        ])
```

```
    q_dot = 0.5 * (Omega @ q)

    return q_dot
```

The result, `q_dot`, is passed to the integrator (e.g. RK4), and the quaternion is renormalized after every step.

7 Conclusion

Quaternion kinematics provides a singularity-free method for propagating spacecraft attitude. The governing equation

$$\boxed{\dot{\boldsymbol{q}} = \frac{1}{2}\,\Omega(\boldsymbol{\omega})\,\boldsymbol{q}}$$

computes the rate of change of orientation from the current angular velocity.
Combined with Euler's rigid body dynamics equation

$$\boldsymbol{I}\dot{\boldsymbol{\omega}} + \boldsymbol{\omega} \times (\boldsymbol{I}\boldsymbol{\omega}) = \boldsymbol{\tau}$$

the quaternion kinematic equation forms the foundation of spacecraft attitude propagation within the ADCS simulator. These two equations constitute the core mathematical model upon which attitude determination, state estimation, and control algorithms will later be built.